Задание 1.

Программа написана на языке Python с использованием библиотеки python-bitcoin-utils и предназначена для программного формирования, подписи и отправки Taproot-транзакций в тестовой сети Bitcoin (Testnet4). На первом этапе скрипт инициализирует окружение для работы с тестовой сетью и на основе заданного приватного ключа в формате HEX генерирует соответствующую пару ключей. Из публичного ключа математически вычисляется Taproot-адрес (тип P2TR, начинающийся с префикса tb1p), который выступает в роли отправителя средств.

Далее программа автоматически обращается к публичному REST API сервиса mempool.space для получения списка непотраченных выходов (UTXO), привязанных к адресу отправителя. Получив данные о доступных монетах, скрипт формирует структуру новой транзакции: в качестве входа указывается найденный UTXO (его хэш и номер выхода), а в качестве выхода — адрес получателя и сумма перевода. При этом сумма перевода рассчитывается как разница между номиналом найденного UTXO и фиксированной комиссией в 1000 сатоши, которая будет взыскана сетью за обработку транзакции.

На завершающем этапе транзакция подписывается с использованием алгоритма Schnorr, что является обязательным требованием для адресов Taproot. Полученная подпись помещается в специальную структуру witness, после чего транзакция сериализуется в HEX-формат. Готовая строка отправляется методом POST на сервис mempool.space для валидации и последующей трансляции в блокчейн. В случае успеха программа получает уникальный идентификатор транзакции (TXID) и выводит в консоль ссылку на блокчейн-эксплорер для отслеживания её статуса.

```
from bitcoinutils.setup import setup
from bitcoinutils.keys import PrivateKey, P2trAddress
from bitcoinutils.transactions import Transaction, TxInput, TxOutput,
TxWitnessInput
import requests


private_key_hex =
'738fcde10f9926f8e445c2a8e72bd749daefe0c982905397988ee206431c1214'
recipient_address =
'tb1pumwese3hl8lq42xk9g8ka6k5mgj6l999d9pap5syqfhrnw5l246q6l9yfu'
FEE = 1000
setup('testnet4')

priv = PrivateKey(secret_exponent=int(private_key_hex, 16))
my_address = priv.get_public_key().get_taproot_address()
my_address_str = my_address.to_string()
print(f"Ваш Taproot адрес (отправитель): {my_address_str}")
```

```
print("Ищем UTXO в сети...")
utxo_url =
f"https://mempool.space/testnet4/api/address/{my_address_str}/utxo"
response = requests.get(utxo_url)

if response.status_code != 200:
    print(f"❌ Ошибка при запросе UTXO: {response.text}")
    exit()

utxos = response.json()
if not utxos:
    print("❌ На этом адресе нет непотраченных монет (UTXO). Подождите
подтверждения или получите монеты снова.")
    exit()

utxo = utxos[0]
utxo_txid = utxo['txid']
utxo_vout = utxo['vout']
utxo_value = utxo['value']

print(f"Найден UTXO: TXID: {utxo_txid}, VOUT: {utxo_vout}, Сумма:
{utxo_value} сатоши")

amount_to_send = utxo_value - FEE

if amount_to_send <= 0:
    print("❌ Недостаточно средств для покрытия комиссии.")
    exit()

txin = TxInput(utxo_txid, utxo_vout)
txout = TxOutput(amount_to_send,
P2trAddress(recipient_address).to_script_pub_key())

tx = Transaction([txin], [txout], has_segwit=True)

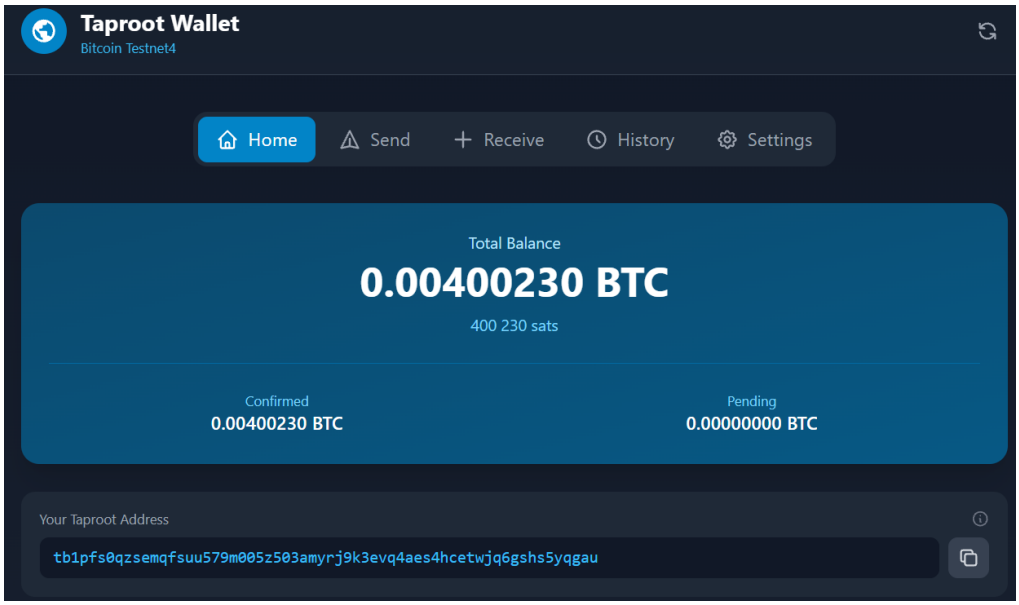
my_script_pub_key = my_address.to_script_pub_key()
signature = priv.sign_taproot_input(
    tx,
    0, # индекс входа
    [my_script_pub_key],
    [utxo_value]
)

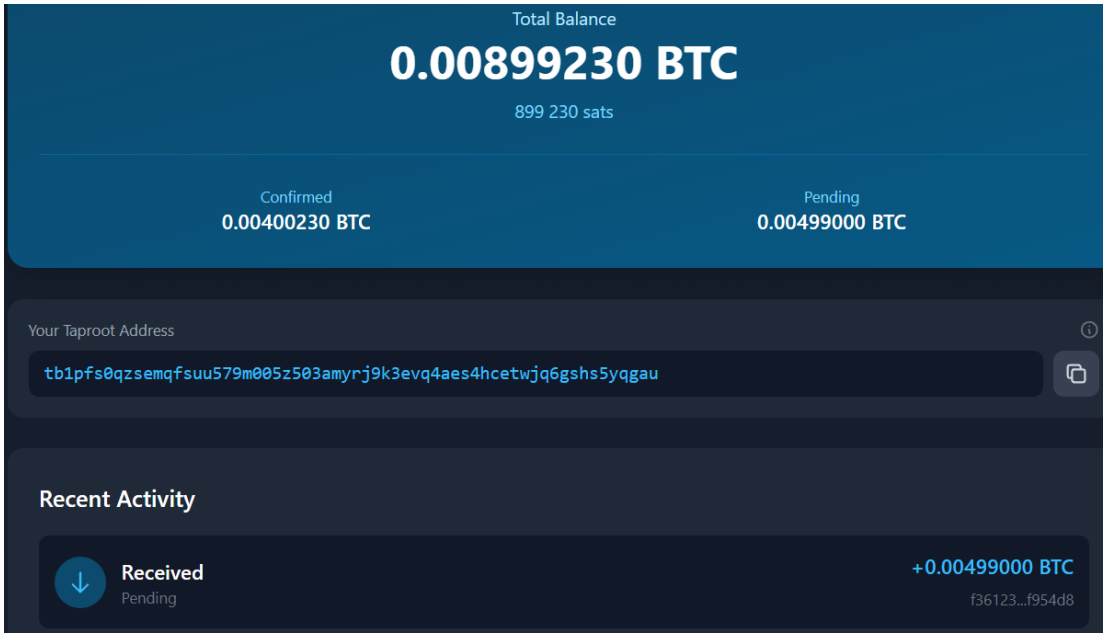
tx.witnesses.append(TxWitnessInput([signature]))

tx_hex = tx.serialize()
print(f"\nГотовая транзакция (hex):\n{tx_hex}")

print("\nОтправка транзакции в сеть...")
url = 'https://mempool.space/testnet4/api/tx'
headers = {'Content-Type': 'text/plain'}
try:
    response = requests.post(url, data=tx_hex, headers=headers)
    response.raise_for_status()
    txid = response.text
    print(f"Транзакция успешно отправлена!")
    print(f"TXID: {txid}")
    print(f"Ссылка для проверки: https://mempool.space/testnet4/tx/{txid}")
except requests.exceptions.RequestException as e:
    print(f"Ошибка при отправке: {e}")
    if response.text:
        print(f"Ответ сервера: {response.text}")
```

Получатель ДО / ПОСЛЕ





Консоль

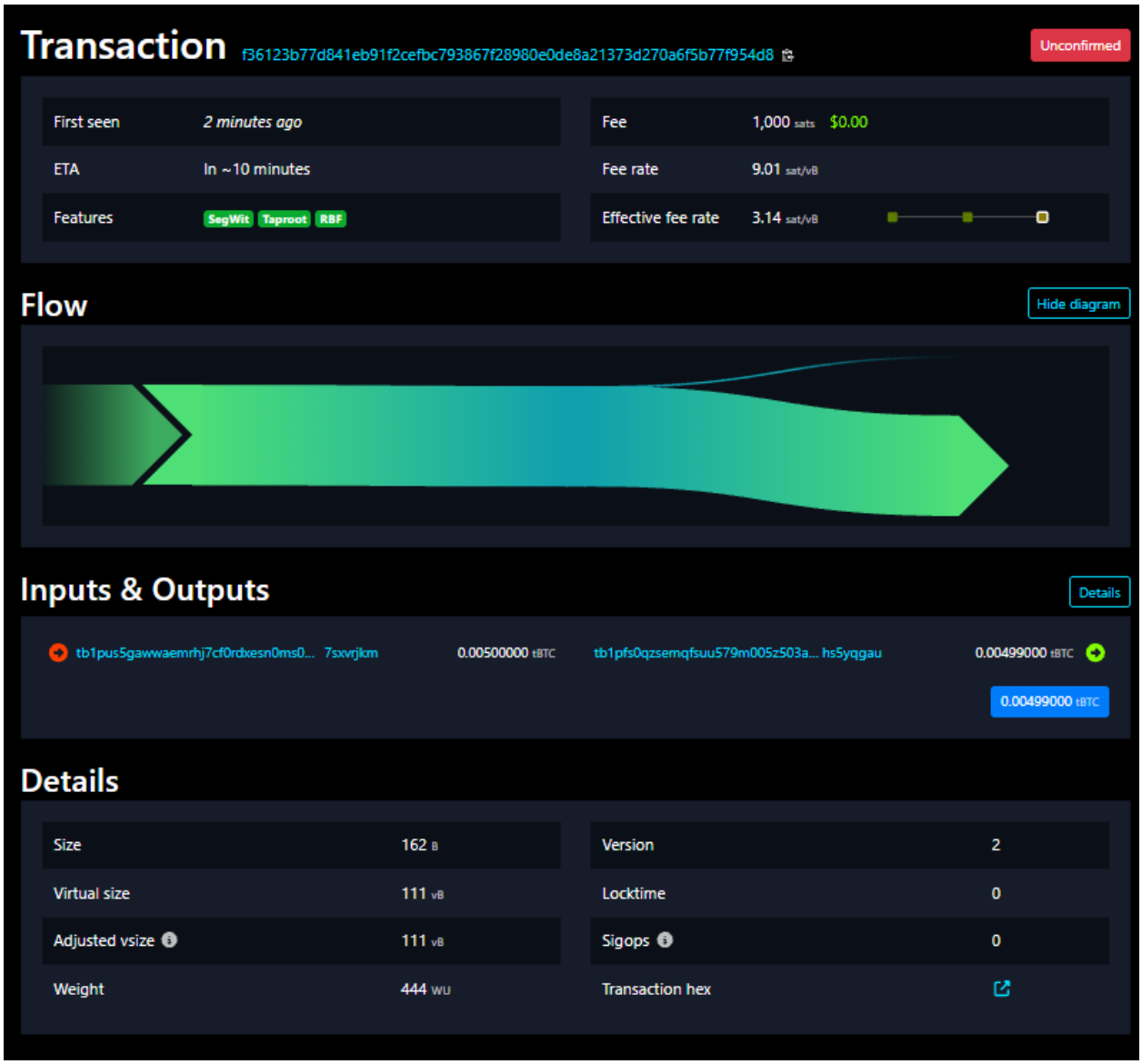
```
D:\Projects\sobes\.venv\Scripts\python.exe "D:\uni\1 sem\blockchain\lab2\send_taproot.py"
Ваш Taproot адрес (отправитель): tb1p5taz43nhxtwmqakmdauazjgsxwdke76effhp5s0ld0che2ynnxls3sywmg
Ищем UTXO в сети...
Найден UTXO: TXID: 8a1170cf1f4fdc32c65ae54526fb8451244f166197d53649a70de1c123046daf, VOUT: 1, Сумма: 500000 сатоши

Готовая транзакция (hex):
02000000000101af6d0423c1e10da74936d59761164f245184fb2645e55ac632dc4f1fcf70118a0100000000fdffffff01389d0700000000002251

Отправка транзакции в сеть...
Транзакция успешно отправлена!
TXID: bba490581dc72854b4649a78ad7164ab586c06670b8da28847162377a24da8ad
Ссылка для проверки: https://mempool.space/testnet4/tx/bba490581dc72854b4649a78ad7164ab586c06670b8da28847162377a24da8

Process finished with exit code 0
```

Транзакция:



Задание 2.

Программа написана на языке Python с использованием библиотеки python-bitcoin-utils и демонстрирует полный цикл работы с мультисигнатурным кошельком (схема 2-из-2) в тестовой сети Bitcoin (Testnet4). На первом этапе скрипт программно генерирует две независимые пары ключей, имитируя двух участников (Алису и Боба). Из их публичных ключей формируется скрипт OP_CHECKMULTISIG, требующий наличия ровно двух подписей для траты средств. Этот скрипт оборачивается в формат P2WSH (Pay-to-Witness-Script-Hash), в результате чего генерируется адрес, начинающийся с префикса tb1q. Поскольку скрипт работает в автоматическом режиме, он делает паузу и ожидает, пока пользователь пополнит сгенерированный адрес через кран (faucet) и нажмет клавишу Enter для продолжения.

После пополнения программа автоматически обращается к публичному API сервиса mempool.space, чтобы найти непотраченный выход (UTXO), привязанный к мультисиг-адресу. Получив данные о транзакции, скрипт формирует новую транзакцию на перевод средств. Ключевой этап — подпись транзакции. Так как адрес является SegWit-адресом, используется метод sign_segwit_input, который учитывает сумму UTXO и хэш скрипта согласно стандарту BIP143. Транзакция подписывается обоими ключами по очереди (сначала Алисой, затем Бобом), что математически доказывает согласие обоих участников на перевод.

Особое внимание в коде уделено формированию структуры witness (свидетеля). Из-за исторического бага в оригинальном коде Bitcoin (операция OP_CHECKMULTISIG ошибочно снимает со стека на один элемент больше), в начало witness-стека принудительно добавляется пустой массив байтов (в коде это реализовано через пустую строку ''). Это требование протокола (BIP 147, правило NULLDUMMY), без которого сеть отклонит транзакцию с ошибкой валидации. В завершение транзакция сериализуется в HEX-формат и отправляется методом POST на сервер mempool.space, который транслирует её в блокчейн, возвращая уникальный идентификатор (TXID) для проверки в эксплорере.

```
from bitcoinutils.setup import setup
from bitcoinutils.keys import PrivateKey, P2wshAddress, P2trAddress
from bitcoinutils.script import Script
from bitcoinutils.transactions import Transaction, TxInput, TxOutput,
TxWitnessInput
import requests

setup('testnet4')

alice_priv = PrivateKey()
bob_priv = PrivateKey()

alice_pub = alice_priv.get_public_key().to_hex()
bob_pub = bob_priv.get_public_key().to_hex()

print(f"Публичный ключ Алисы: {alice_pub}")
print(f"Публичный ключ Боба: {bob_pub}")

multisig_script = Script([2, alice_pub, bob_pub, 2, 'OP_CHECKMULTISIG'])

p2wsh_addr = P2wshAddress.from_script(multisig_script)
multisig_address = p2wsh_addr.to_string()
print(f"\n✅ Ваш 2-из-2 мультисиг-адрес (P2WSH): {multisig_address}")

print("\n" + "="*60)
print("ШАГ 1: Пополните этот адрес через кран (faucet).")
print("Скопируйте адрес выше и запросите тестовые монеты (например, 50000
сатоши).")
print("Ссылка на кран: https://mempool.space/testnet4/faucet")
```

```
print("="*60)

input("Нажмите Enter после того, как монеты появятся на адресе (подождите 1-2
минуты)...")

print("\nИщем UTXO на мультисиг-адресе...")
utxo_url =
f"https://mempool.space/testnet4/api/address/{multisig_address}/utxo"
response = requests.get(utxo_url)

if response.status_code != 200:
    print(f"❌ Ошибка при запросе UTXO: {response.text}")
    exit()

utxos = response.json()
if not utxos:
    print("❌ На адресе нет монет. Убедитесь, что вы пополнили его, и
подождите пару минут.")
    exit()

utxo = utxos[0]
utxo_txid = utxo['txid']
utxo_vout = utxo['vout']
utxo_value = utxo['value']
print(f"Найден UTXO: TXID: {utxo_txid}, VOUT: {utxo_vout}, Сумма:
{utxo_value} сатоши")

recipient_address =
'tb1pumwese3hl8lq42xk9g8ka6k5mgj6l999d9pap5syqfhrnw5l246q6l9yfu'
fee = 1000
amount_to_send = utxo_value - fee

if amount_to_send <= 0:
    print("❌ Недостаточно средств для покрытия комиссии.")
    exit()

txin = TxInput(utxo_txid, utxo_vout)

recipient_address = (
    'tb1pumwese3hl8lq42xk9g8ka6k5mgj6l999d9pap5syqfhrnw5l246q6l9yfu'
)

txout = TxOutput(
    amount_to_send,
    P2trAddress(recipient_address).to_script_pub_key()
)
tx = Transaction([txin], [txout], has_segwit=True)

print("\nПодписываем транзакцию двумя ключами...")

signature_alice = alice_priv.sign_segwit_input(tx, 0, multisig_script,
utxo_value)
signature_bob = bob_priv.sign_segwit_input(tx, 0, multisig_script,
utxo_value)

witness = TxWitnessInput(['', signature_alice, signature_bob,
multisig_script.to_hex()])
tx.witnesses.append(witness)

tx_hex = tx.serialize()
print(f"\nГотовая транзакция (hex):\n{tx_hex}")

print("\nОтправка транзакции в сеть...")
```

```python
url = 'https://mempool.space/testnet4/api/tx'
headers = {'Content-Type': 'text/plain'}
try:
    response = requests.post(url, data=tx_hex, headers=headers)
    response.raise_for_status()
    txid = response.text
    print(f"✅ Транзакция успешно отправлена!")
    print(f"TXID: {txid}")
    print(f"Ссылка для проверки: https://mempool.space/testnet4/tx/{txid}")
except requests.exceptions.RequestException as e:
    print(f"❌ Ошибка при отправке: {e}")
    if response.text:
        print(f"Ответ сервера: {response.text}")
```

Консоль

```
Публичный ключ Алисы: 03bb8e01596e34c2820e8591677b1fb2ab2b70453e627d3c228c6880a8f30b8675
Публичный ключ Боба: 0243dfc4df005b4a1c1bbe679d48965c6870b05cc218df76dba26f7d2337205dc8

✅ Ваш 2-из-2 мультисиг-адрес (P2WSH): tb1qvt9uaqkqrs3cwpkhr5ls3mhwkqptecxvvqzjdggf2rc4q7msjctqu0z44r

============================================================
ШАГ 1: Пополните этот адрес через кран (faucet).
Скопируйте адрес выше и запросите тестовые монеты (например, 50000 сатоши).
Ссылка на кран: https://mempool.space/testnet4/faucet
============================================================
Нажмите Enter после того, как монеты появятся на адресе (подождите 1-2 минуты)...

Ищем UTXO на мультисиг-адресе...
Найден UTXO: TXID: 772071dc8830158b9a74aa992c66f4ffaffb6431135a89db0ab1c5b1e7e22afb, VOUT: 0, Сумма: 99230 сатоши

Подписываем транзакцию двумя ключами...

Готовая транзакция (hex):
02000000000101fb2ae2e7b1c5b10adb895a133164fbafff4662c99aa749a8b153088dc7120770000000000fdffffff01b67f010000000000220020 62cbce82c01c238706d71d3f08eeeeb002bce

Отправка транзакции в сеть...
✅ Транзакция успешно отправлена!
TXID: 96f3b66e5623018d45ca708e44915f0e43109e7f1baac27b9703b19d3fb0c190
Ссылка для проверки: https://mempool.space/testnet4/tx/96f3b66e5623018d45ca708e44915f0e43109e7f1baac27b9703b19d3fb0c190

Process finished with exit code 0
```

Транзакция

Transaction

96f3b66e5623018d45ca708e44915f0e43109e7f1baac27b9703b19d3fb0c190

10 confirmations

Timestamp	2026-11-05 18:43:47 *(just now)*	Fee	1,000 sats $0.00
Features	SegWit Taproot RBF	Fee rate	6.71 sat/vB Overpaid 7x

Flow

Hide diagram



Inputs & Outputs

Details

tb1qvt9uaqkqrs3cwpkhr5ls3mhwkqpt... tqu0z44r Multisig 2 of 2	0.00099230 tBTC	tb1qvt9uaqkqrs3cwpkhr5ls3mhwkq... tqu0z44r	0.00098230 tBTC
			0.00098230 tBTC

Details

Size	314 B	Version	2
Virtual size	149 vB	Locktime	0
Adjusted vsize	149 vB	Sigops	2
Weight	596 WU	Transaction hex	